

Exemplo - Módulo 4: LoRA e PEFT

Disciplina: Processamento de Dados e Fine-Tuning de Modelos · Prof. Ahirton Lopes, Ph.D.

Missão Prática #04 - solução de referência

Resolvida com o caso real da Amplitude Seguros: a parceria regional de orçamentos de oficina que o Módulo 4.1 apresentou, usando os números reais dos treinos rodados nos Módulos 4.2, 4.3 e 4.4.

Passo 1 - Caso escolhido e NPV

Caso: parceria regional da Amplitude Auto (Sul e Nordeste), mesma tarefa aprovada pelo Módulo 1.3 (extrair segurado, placa e valor), mas com volume de 400 orçamentos por mês, contra 8.000 por mês da Amplitude Auto nacional.

NPV em 24 meses, custo de GPU alugada (R\$ 2.400 por job, valor de referência do Módulo 1.3): R\$ -2.040,99, nunca atinge breakeven no horizonte. NPV com custo de treinamento local (~R\$ 0, hardware já existente): R\$ 359,01, breakeven já no mês 1. Mesmo volume, mesmo custo por chamada, resultado oposto: o formato do custo fixo decide.

Passo 2 - Treinos LoRA locais, três postos

Dataset real de 200 exemplos do Módulo 3.2 (120 Amplitude Auto, 80 Saúde Empresarial), convertido do formato Gemini (contents/role/parts) pro formato messages (role/content), dividido em 157 exemplos de treino, 30 de validação, 13 de teste.

Rank 4: 0,074% do modelo treinável (3,408M parâmetros), val loss 4,752 → 1,246, pico de memória 10,787 GB, adaptador de 13 MB.

Rank 8 (mesmo do Módulo 4.2): 0,147% (6,816M), val loss 4,752 → 0,895, pico de memória 10,833 GB, adaptador de 26 MB.

Rank 16: 0,295% (13,631M), val loss 4,752 → 0,725, pico de memória 10,930 GB, adaptador de 52 MB. Retorno decrescente medido: rank 4→8 melhora 28,17% o val loss, rank 8→16 melhora só 18,99%, mesma dobra de parâmetros.

(medido em iters=20 - o mesmo ponto que o Módulo 4.2 mostrou subtreinado; a ordem entre os postos deve se manter, mas os percentuais exatos podem mudar até convergência)

Passo 3 - Exemplo difícil, construído de propósito

Caso novo, nunca visto no treino: orçamento da "Oficina Boa Vista Reparos Automotivos", com dois valores distratores antes do valor certo, valor das peças R\$ 1.850,00, valor da mão de obra R\$ 970,00, e só depois o valor total do orçamento, R\$ 2.820,00, o único campo correto pra extrair.

Resultado: sem adaptador, o modelo base entra em raciocínio livre e não termina dentro do limite de tokens. Com qualquer um dos três postos, quatro, oito ou dezesseis, a saída bate exatamente com o gabarito, segurado Ricardo Alves Monteiro, placa JBR-9021, valor 2.820, ignorando corretamente os dois valores distratores. Nenhum dos três postos errou, mesmo no caso difícil.

Passo 4 - Decisão final: LoRA ou full fine-tuning

Full fine-tuning rodado nas mesmas 16 camadas que o LoRA usa por padrão: 22,567% do modelo treinável (1.044,510M parâmetros), val loss final 0,612, pico de memória 15,338 GB, checkpoint de ~1.992 MB.

Comparado ao melhor LoRA (rank 16, val loss 0,725), o ganho real do full fine-tuning é de 15,59%. Frente ao LoRA rank 8, a configuração padrão, full usa 153,5 vezes mais parâmetros, ~42% mais memória de pico, e checkpoint 76,6 vezes maior. Com limiar de decisão de 20% (implementado em valeAPenaFullFineTuning), esse ganho não justifica o custo extra. E no exemplo difícil do Passo 3, full fine-tuning acerta exatamente igual aos três postos de LoRA: o teto de qualidade é real em val loss, mas não aparece em comportamento observável nesta tarefa.

Decisão documentada: LoRA rank 8 é a escolha certa pra essa parceria regional. O ganho de qualidade do rank 16 sobre o rank 8 (18,99%) já é pequeno; o ganho adicional do full fine-

tuning sobre o rank 16 (15,59%) é ainda menor e custa muito mais. Ver `local-lora-training-tool.js` (Módulo 4.2, 16 testes), `lora-rank-tradeoff-tool.js` (e o par em Python, `lora_rank_tradeoff_tool.py`; Módulo 4.3, 13 testes, fonte dos números de retorno decrescente entre postos) e `full-vs-lora-tradeoff-tool.js` (e o par em Python, `full_vs_lora_tradeoff_tool.py`; Módulo 4.4, 8 testes), testes automatizados que confirmam cada número acima.

Ressalva: os dois treinos (LoRA e full) pararam em `iters=20`, o mesmo ponto que o Módulo 4.2 mediu como subtreino (curva estendida a 120 iterações aponta a melhor iteração em 90). A decisão acima vale pra esse orçamento de tempo; antes de fixar em produção, repitam a comparação até o val loss estabilizar.

Nota de curadoria

Reparem no padrão que se repete em cada passo desta atividade: a decisão nunca foi tomada por regra de bolso, "LoRA é sempre melhor" ou "mais posto é sempre melhor", mas sim tomada rodando processos reais e medindo o resultado contra um critério explícito. É a mesma disciplina do NPV do Módulo 1.3, aplicada agora a hiperparâmetros de treino. Ao fazer a atividade de vocês, resistam à tentação de pular a medição e já chutar a resposta: o valor do módulo inteiro está em mostrar que a resposta certa muda dependendo do caso, e só os números reais revelam qual é.

Ahirton Lopes · Fine-Tuning Toolkit - UNIPDS: Processamento de Dados e Fine-Tuning de Modelos
Prof. Ahirton Lopes, Ph.D. - GDE AI, Microsoft MVP, Senior Manager